

Извлечение данных с WEB-страниц. Пакет rvest.

4.1. Цель работы:

Научиться работать извлекать информацию с WEB-страниц с помощью инструментов языка R.

4.2. Общие сведения

Иногда возникает необходимость получить данные с веб-страниц и сохранить их в структурированном виде, для нашего курса это особенно актуально.

Инструменты веб-скрейпинга ([web scraping](#)) разрабатываются для извлечения данных с веб-сайтов. Эти инструменты бывают полезны тем, кто пытается получить данные из Интернета. Веб-скрейпинг — это технология, позволяющая получать данные без необходимости открывать множество страниц и заниматься копипастом. Эти инструменты позволяют вручную или автоматически извлекать новые или обновленные данные и сохранять их для последующего использования. Например, с помощью инструментов веб-скрейпинга можно извлекать наборы данных для дальнейшего анализа.

4.2.1. Возможные сценарии использования инструментов веб-скрейпинга:

- Сбор данных для маркетинговых исследований;
- Извлечение контактной информации (адреса электронной почты, телефоны и т.д.) с разных сайтов для создания собственных списков поставщиков, производителей или любых других лиц, представляющих интерес.
- Загрузка решений со StackOverflow (или других подобных сайтов с вопросами-ответами) для возможности оффлайн чтения или хранения данных с различных сайтов — тем самым снижается зависимость от доступа в Интернет.
- Поиск работы или вакансий.
- Отслеживание цен на товары в различных магазинах.

4.2.2. Постановка проектной задачи

Необходимо провести экспертный анализ документов, доступных только по запросам на веб портале и по относящимся к исходной задаче составить аналитический отчет. Звучит просто если бы не следующие нюансы:

1. БД документов недоступна напрямую.
2. Никакого API к portalу не существует.
3. Документы по прямой ссылке нельзя адресовать, только через механизм создания сессионного подключения и поиска.
4. Полный текст документа недоступен, его показ формируется JS скриптами для «постраничного» листания.
5. По каждому из подходящих документов необходимо, опираясь на plain text 1-ой страницы сделать реферативную сводку по его атрибутам.
6. Документов ~100 тыс, реально относящихся к задаче ~0.5%, времени на первый релиз ~ 1 неделя.

Копирование таблиц или списков с веб-сайтов – занятие, скучное и монотонное. К тому же этот процесс подвержен ошибкам и трудно воспроизводим. К счастью, существуют пакеты для Python и R, позволяющие автоматизировать выполнение подобных задач. В этой лабораторной мы изучим и опробуем библиотеку **rvest**.

Решение задачи предполагает уверенное знание дерева DOM и CSS – селекторов.

Предварительно необходимо загрузить библиотеку:

```
install.packages("rvest")  
  
library(rvest)
```

Для работы может также понадобится браузеры отладчик HTML-страниц, также может помочь плагин для Chrom: <http://selectorgadget.com/>.

4.3. Краткий обзор функций библиотеки rvest

Наиболее важные функции в rvest:

- **read_html()** - Создание документа html из url, файла на диске или строки, содержащей html;
- Выделение фрагментов документа с помощью CSS-селекторов **html_nodes()** (`doc, "table td"`) (возможно, использование селекторов xpath с `html_nodes(doc, xpath = "///table//td")`).
- Извлечение компоненты с **html_tag()** (имя тега), **html_text()** (весь текст внутри тега), **html_attr()** (содержимое одного атрибута) и **html_attrs()** (все атрибуты).
- (Вы можете также использовать rvest с файлами XML распарсить в **XML()**, а затем извлекать компоненты с помощью **xml_node()**, **xml_attr()**, **xml_attrs()**, **xml_text()** и **xml_tag()**.)
- Разбор таблиц в рамках данных с **html_table()**.
- Извлекать, изменять и отправлять формы с **html_form()**, **set_values()** и **submit_form()**.
- Обнаруживать и устранять проблемы с кодировкой **guess_encoding()** и **repair_encoding()**.
- Навигация по сайту если вы в браузере с **html_session()**, **jump_to()**, **follow_link()**, **back()**, **forward()**, **submit_form()** и так далее. Ссылки на документацию:

<https://www.rdocumentation.org/packages/rvest/versions/0.3.2>
<https://cran.r-project.org/web/packages/rvest/rvest.pdf>

4.4. Практические примеры

Пример 4.1:

Задача:

В качестве примера извлечем информацию о гастрономических турах России. Для этого обратимся к ресурсу: <https://www.kp.ru/russia/tury-po-rossii/gastronomicheskie/>

На этой странице после заголовка расположены блоки, содержащие описание тура, слева-фото. Каждое фото также служит ссылкой на другую страницу, содержащую развернутое описание гастрономического тура:

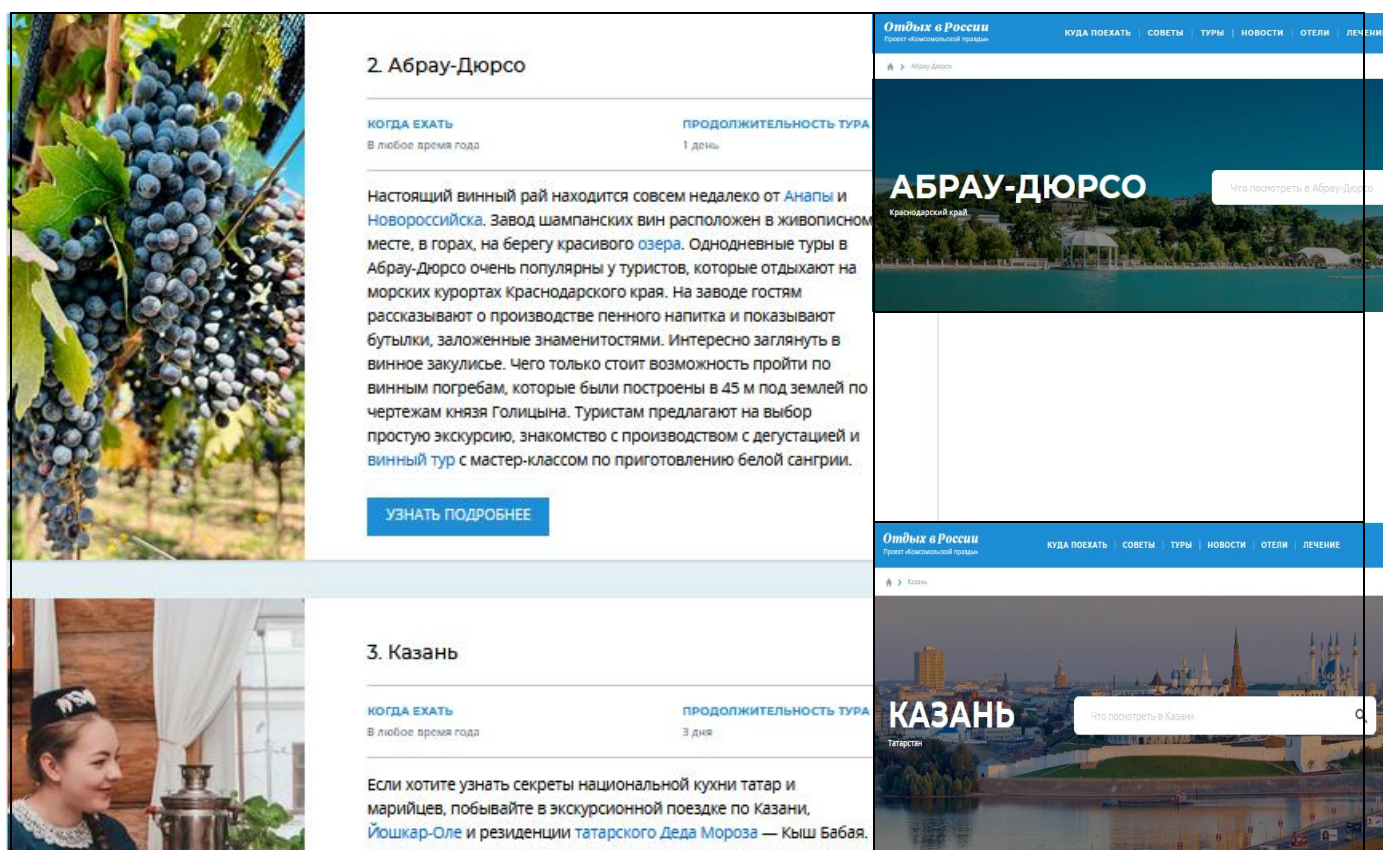


Рис.4.1. Постановка задачи поиска на странице. Структура элементов и переходы

При сборе данных с веб-сайтов труднее всего определить, какие фрагменты HTML-кода необходимо извлечь. Обычно веб-страница представляет собой сложную структуру, состоящую из вложенных объектов, называемую объектной моделью документа (documentobject model, DOM). Соответственно, вы должны выяснить, какие фрагменты DOM вам нужны.

Чтобы сделать это, необходимо исследовать код веб-страницы с помощью инструментов разработчика, предоставляемых вашим браузером. Если вы используете Chrome или Firefox, открыть инструменты разработчика можно нажатием F12 (или Cmd + Opt + I для Mac), если вы используете Safari – Cmd + Opt + I.

Чтобы следить за ходом изложения, откройте страницу со списком гастрономических туров, которая используется в нашем примере.

Нажав [Ctrl+Shift+i] в Chrome, вы увидите нечто подобное тому, что изображено на рисунке ниже. Обратите особое внимание на инструмент для выбора элементов, обведенный красным цветом, и убедитесь в том, что у вас открыта вкладка Elements (элементы).

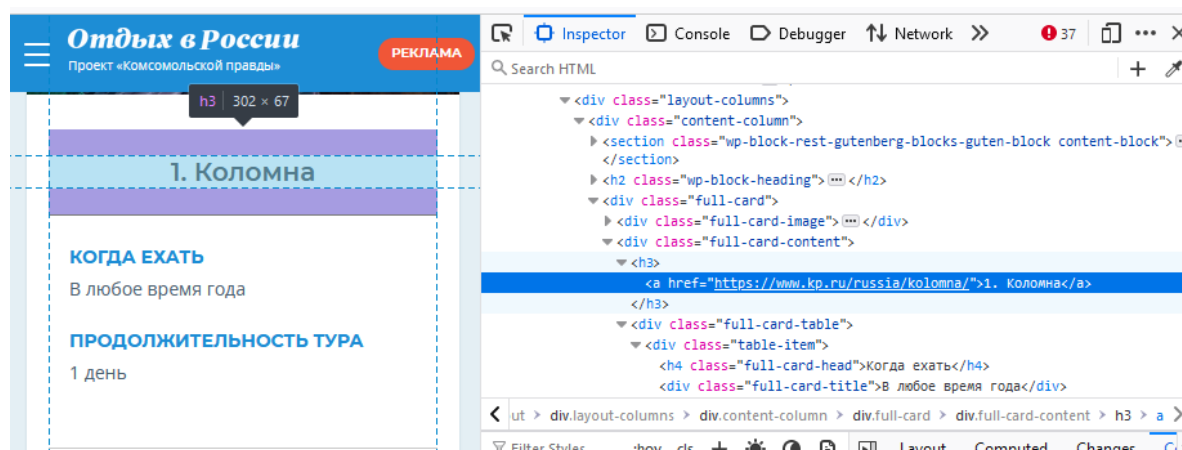


Рис.4.2. Выделение необходимого элемента и нахождение CSS – селектора для него.

Функция `html_nodes` извлекает весь узел из DOM, а затем функция `html_text` позволяет нам извлечь текст из узла. Обратите внимание на использование конвейера `%>%`, который передает результат выполнения функции `html_nodes` в функцию `html_text`.

```
url2<-read_html("https://www.kp.ru/russia/tury-po-rossii/gastronomicheskie")
# Найдем на странице и зададим имя селектора, который нам позволит выбрать города
selector_name<-"full-card-content>h3>a"
# Извлечённые имена городов по конвейеру извлечем текст из узлов и запишем их в массив:
tnames<-html_nodes(url2, selector_name) %>% html_text()%>%as.array(); fnames

> tnames
[1] "1. коломна"          "2. Абрау-Дюрсо"      "3. Казань"
[4] "4. Тверь"           "5. Астрахань"        "6. Вологда"
[7] "7. карелия"         "8. Смоленск"        "9. Калининград"
[10] "10. Удмуртия"        "11. Дагестан"        "12. Байкал"
[13] "13. Архангельская область" "14. Владивосток"    "15. Ингушетия"
[16] "16. Красноярск"     "17. Мурманск"       "18. Ростов-на-Дону"
[19] "19. Адыгея"         "20. Ярославль"
```

То же можно получить немного другим набором команд:

```
# URL for the visit website
url1<-'https://www.kp.ru/russia/tury-po-rossii/gastronomicheskie'
download.file(url1, destfile = "scrapedpage1.html", quiet=TRUE)
content1 <- read_html("scrapedpage1.html")
# Найдем на странице и зададим имя селектора, который нам позволит выбрать города
selector_name<-"full-card-content>h3>a"
# Извлечённые имена городов по конвейеру извлечем текст из узлов и запишем их в массив:
fnames<-html_nodes(content1, selector_name) %>% html_text()%>%as.array();fnames
```

Что делает каждая команда, а также что это за оператор канала (pipe)?

1. **read_html** — мы используем его для чтения HTML, и, по сути, он предоставляет URL-адрес и возвращает вам HTML-документ или исходный код целевого URL-адреса.
2. **html_nodes** — Учитывая исходный код HTML, извлекает фактические элементы, которые мы хотим получить.
3. **html_text** — извлекает текст из указанных с помощью селекторов тегов.
4. **Pipe Оператор(%>%)** — это часть библиотеки *Deer Wire*, которая значительно упрощает программирование. Все, что находится слева от `%>%`, вычисляется, принимает результат и передает его в качестве первого аргумента функции, расположенной после `%>%`. Итак, довольно просто и очень полезно.

Усложним задачу

Предположим, нам нужно получить информацию со всех страниц, на которые ссылается кнопка «Узнать подробнее» (рис.4.3)

Тогда, применив инструмент для выбора элементов, находим селекторы, по которым можно найти адреса соседних страниц:

```
<a class="full-card-button" href="https://www.kp.ru/russia/kolomna/">
```

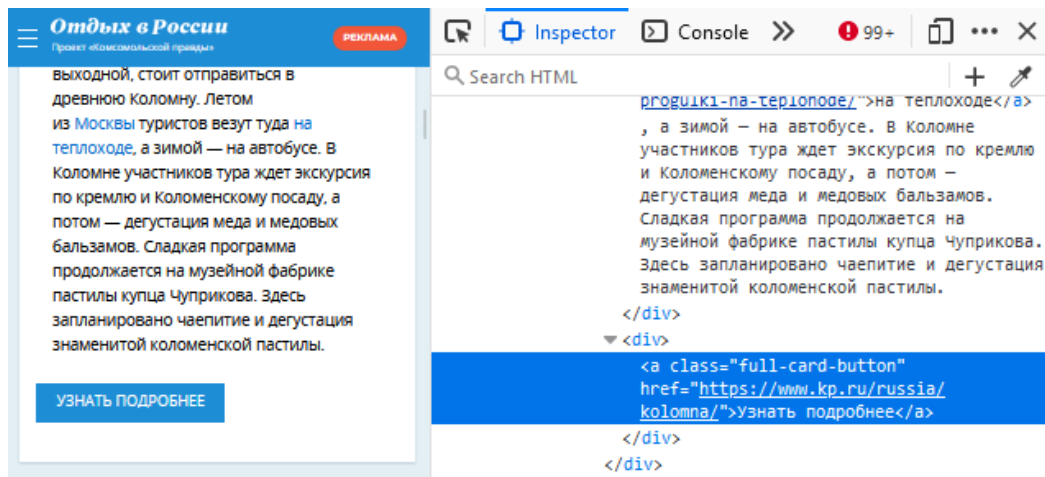


Рис.4.3. Ищем переход на следующие страницы

Найдем далее все ссылки на соседние страницы, используя класс ссылки и, обращаясь к атрибуту **href** найденного элемента:

```
selector_a_name<-"full-card-button"
fnames_addr<-html_nodes(url2, selector_a_name) %>% html_attr("href"); fnames_addr
```

```
[1] "https://www.kp.ru/russia/kolomna/"
[2] "https://www.kp.ru/russia/abrau-dyurso/"
[3] "https://www.kp.ru/russia/kazan/"
[4] "https://www.kp.ru/russia/tver/"
[5] "https://www.kp.ru/russia/astrahan/"
[6] "https://www.kp.ru/russia/vologda/"
[7] "https://www.kp.ru/russia/kareliya/"
[8] "https://www.kp.ru/russia/smolensk/"
```

...

Теперь мы хотим составить список всех достопримечательностей всех найденных городов. Заметим, что для перехода на страницу с достопримечательностями используется кнопка меню «Интересные места» и к URL дописывается "dostoprimechatelnosti/", везде, кроме Абрау-Дюрсо, там надо дописать "#places".

Сделаем это добавление функцией **paste0()** вот так:

```
fnames_addr<-html_nodes(url2, selector_a_name) %>% html_attr("href")
#Приклеим ко всем ссылкам путь к странице "Интересные места"
fnames_addr1<-paste0(fnames_addr, "dostoprimechatelnosti/")
fnames_addr1[2]<-paste0(fnames_addr[2], "#places")
```

Определим список достопримечательностей для Карелии, для этого определим селектор, которой поможет нам найти имя достопримечательности:

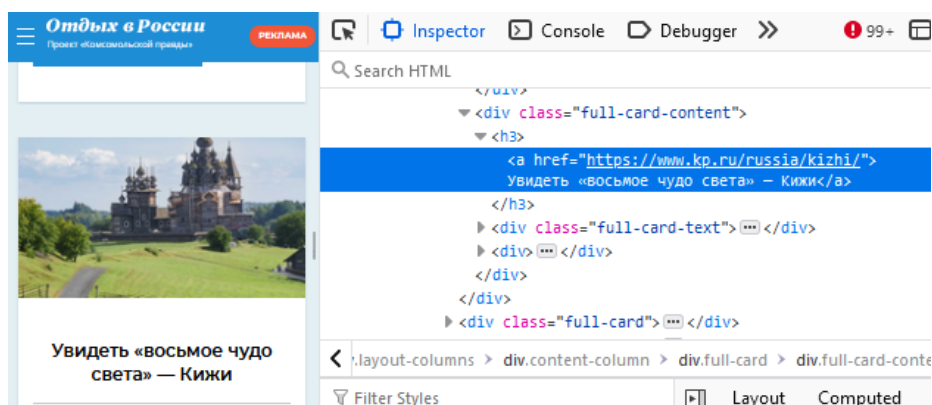


Рис.4.4. Ищем название достопримечательности


```
url_sub<-read_html("https://www.kp.ru/russia/kareliya/")
sub_selector_name<-"full-card-content>h3>a"
> f1_address<-html_text(html_nodes(url_sub, sub_selector_name),trim=TRUE)
# так тоже можно
# f1_places<-html_text(html_nodes(url_sub, sub_selector_name),trim=TRUE)
> f1_address
> f1_places
[1] "погулять по столице Карелии – Петрозаводску"
[2] "побывать на острове-монастыре Валаам"
[3] "увидеть «восьмое чудо света» – Кижь"
[4] "проехать на лодке вдоль мраморных скал «Рускеала»"
[5] "Отель «Фрегат» 4*"
```

Необходимо извлечь достопримечательности всех городов.

Пример 4.2. Чтение данных из таблицы

Обратимся к ресурсу https://ru.wikipedia.org/wiki/Медальный_зачёт_на_зимних_Олимпийских_играх_2022. На данной странице имеются две таблицы. Первые две строки скрипта (ниже) уже не требуют комментариев. В результате их работы будет сформирован список **nodes** из двух элементов (xml_nodeset), каждый из которых содержит таблицу.

```
install.packages("rvest")
library(rvest)

url =
'https://ru.wikipedia.org/wiki/Медальный_зачёт_на_зимних_Олимпийских_играх_2022'
download.file(url, destfile = "scrapedpage.html", quiet=TRUE)
content <- read_html("scrapedpage.html")
nodes = html_nodes(content, 'table'); nodes

df2 = html_table(nodes[[2]])%>%as.data.frame()
colnames(df2)<-df2[1,]
df2<-df2[2:31,]
fix(df2)
```

Последняя строка скрипта указывает, что необходимо прочитать список **nodes**, как отдельный элемент с помощью команды **html_table**, и при этом, преобразовать элемент списка в **data.frame**.

Получим: (фрагмент таблицы)

```
df2
```

Место		Страна	Золото	Серебро	Бронза
2	1	Норвегия Норвегия	16	8	13
3	2	Германия Германия	12	10	5
4	3	Китай Китай	9	4	2
5	4	США США	8	10	7
6	5	Швеция Швеция	8	5	5
7	6	Нидерланды Нидерланды	8	5	4
8	7	Австрия Австрия	7	7	4
9	8	Швейцария Швейцария	7	2	5
...					
31	Всего	Всего	109	109	109

Названия страны задриваются, но это не сильно мешает анализу. Попробуйте найти решение проблемы самостоятельно, работая со строками.

Скриншот с сайта:

Неофициальный медальный зачёт [править | править код]

Отсортирован по количеству золотых медалей, при равном количестве золотых — по количеству серебряных, при равных количествах золотых и серебряных — по количеству бронзовых.

Жирным выделено наибольшее количество медалей в своей категории; страна-организатор также выделена.

Место ↕	Страна ↕	Общее количество медалей			Всего ↕
		Золото ↕	Серебро ↕	Бронза ↕	
1	 Норвегия	16	8	13	37
2	 Германия	12	10	5	27
3	 Китай	9	4	2	15
4	 США	8	10	7	25
5	 Швеция	8	5	5	18
6	 Нидерланды	8	5	4	17
7	 Австрия	7	7	4	18
8	 Швейцария	7	2	5	14
9	 ОКР	6	12	14	32
10	 Франция	5	7	2	14
11	 Канада	4	8	14	26

4.5. Задания к лабораторной работе

Задание 1. В ходе лабораторной работы, необходимо собрать информацию об уровне жизни стран мира из таблиц сайта https://www.numbeo.com/quality-of-life/rankings_by_country.jsp?title=2021 с 2014 по текущий год:

Для дополнительной информации по Рейтингу стран по уровню жизни можно использовать ссылку

<https://tyulyagin.ru/ratings/rejting-stran-mira-po-urovnyu-zhizni-2021.html>

Это оценка общего качества жизни с использованием эмпирической формулы, которая учитывает:

- индекс покупательной способности (чем выше, тем лучше),
- индекс загрязнения (чем ниже, тем лучше),
- отношение цены на жилье к доходу (ниже, лучше),
- индекс прожиточного минимума (чем ниже, тем лучше),
- индекс безопасности (чем выше, тем лучше),
- индекс медицинского обслуживания (чем выше, тем лучше),
- индекс времени движения на дороге (чем ниже, тем лучше)
- климатический индекс (чем выше, тем лучше).

1. Каждый студент должен взять 5 стран (по варианту) .

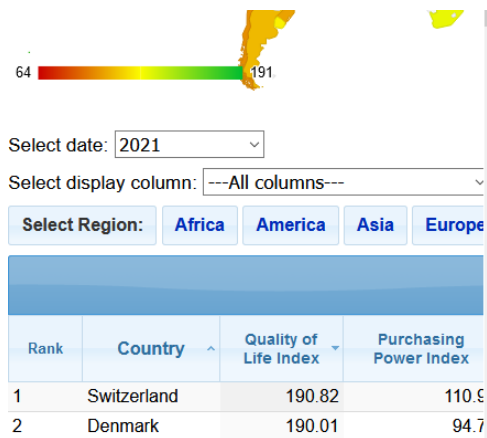
2. Составить data.frame (возможно для каждой страны) так, чтобы иметь возможность **проанализировать с помощью графиков изменение рейтингов для всех 10 показателей для всех своих 5-ти стран, прокомментировать в отчете результат.**

Необходимо нарисовать на одном и том же графике рейтинг всех 5 стран, проанализировать результат, анализ словесно отразить в отчете. Проанализировать изменение во времени всех показателей указанных стран, подобрать наилучший (с вашей точки зрения) способ визуализации.

3. С одной из страниц (по варианту) : вариант брать по формуле №пп % №ссылки +1.

4. С одной из страниц (по варианту) : вариант брать по формуле №пп % №ссылки + 1.

- 1) <https://kudago.com/spb/list/33-luchshih-muzeya-peterburga/>
- 2) https://tonkosti.ru/Музеи_Санкт-Петербурга



- 3) https://tonkosti.ru/Достопримечательности_Крыма
- 4) <https://www.sputnik8.com/ru/moscow> (начало, окончание, длительность, цена)
- 5) https://ru.wikipedia.org/wiki/Список_музеев_Москвы (на соседней странице прочитать адрес музея, все записать в единую таблицу – название музея и его адрес)
- 6) <https://www.afisha.ru/msk/museum/> на соседней странице прочитать режим работы, цены и адрес музея, все записать в единую таблицу)
- 7) <https://www.culture.ru/museums/institutes/location-krasnodarskiy-kray-krasnodar> (-//-)
- 8) https://ru.wikipedia.org/wiki/Список_музеев_Ростовской_области (-//-)
- 9) <https://www.afisha.ru/krasnodar/museum/>
- 10) <https://ru.wikipedia.org/wiki/Калининград> (-//-)

собрать информацию в data.frame, которая содержала бы: Название музея, его адрес, описание музея (если есть) и ссылку для перехода при клике на фото / ссылке на музей.

Варианты: Выполняем скрепинг по тем же вариантам для стран, что и в ЛР3.

Задание 2. С помощью языка Python собрать информацию о недвижимости с любого сайта недвижимости в районе вашего проживания (не менее 200 записей). Пример нужно доработать, чтобы получить характеристики квартиры, адрес, цену, фото.

Пример скрипта:

```
import requests
from bs4 import BeautifulSoup
import csv
import time
import random

count = 1
# Список для хранения данных
apartments = []
# URL страницы для скраппинга

# Заголовки для имитации запроса от браузера

import random

# Список прокси (HTTP/HTTPS/SOCKS)
PROXIES = [
    "http://45.67.12.34:8000",
    "http://193.123.44.56:3128",
    "socks5://user:pass@proxy_ip:1080"
]

USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36",
```



```

        "Mozilla/5.0 (iPhone; CPU iPhone OS 15_0 like Mac OS X) AppleWebKit/605.1.15
        (KHTML, like Gecko) Version/15.0 Mobile/15E148 Safari/604.1"
    ]

    # Выбираем случайный User-Agent
    random_ua = random.choice(USER_AGENTS )
    headers = {"User-Agent": random_ua}

    #headers = {
    #   "User -Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    #   (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36"
    #}
    while count <= 5:
        url =
        "https://krasnodar.cian.ru/cat.php?deal_type=sale&engine_version=2&offer_type=flat&p="+str(count)+"&region=4820"
        # Выполнение GET-запроса

        headers = {"User-Agent": random.choice(USER_AGENTS)}

        response = requests.get(url, headers=headers)

        # Проверка успешности запроса
        if response.status_code == 200:
            print("Успех! Доступ получен.")
            # Парсинг HTML-кода страницы
            soup = BeautifulSoup(response.text, 'html.parser')
            # Находим все объявления
            ads = soup.find_all('div', class_='93444fe79c--card--ibP42_93444fe79c--wide--gEKNN')

            # Извлечение информации из каждого объявления
            for ad in ads:
                title = ad.find('div', class_='93444fe79c--row--kEHOK').get_text(strip=True) if
                ad.find('div', class_='93444fe79c--row--kEHOK') else 'Нет заголовка'
                price = ad.find('div', class_='93444fe79c--container--aWzpE').get_text(strip=True)
                if ad.find('div', class_='93444fe79c--container--aWzpE') else 'Цена не указана'
                print(price)

                apartments.append({
                    'title': title,
                    'price': price
                })

            value = random.random()
            scaled_value = 1 + (value * (9 - 5))
            time.sleep(scaled_value)

        else:
            print(f"Ошибка при запросе: {response.status_code}")
            break
    count += 1

```

```
# Выводим собранные данные
for item in apartments:
    print(f"Заголовок: {item['title']}, Цена: {item['price']}")
    # Запись данных в CSV-файл
    with open('avito_apartments.csv', mode='w', newline="", encoding='utf-8') as file:
        writer = csv.DictWriter(file, fieldnames=['title', 'price'])
        writer.writeheader() # Запись заголовков
        for item in apartments:
            writer.writerow(item) # Запись строк с данными

print("Данные успешно записаны в avito_apartments.csv")
```